When the Model Says It Checked: A Finite Local-Effect Audit of Model-Internal Verification

Zewen Liu
Independent Researcher
17353895263@163.com

Abstract

As agent models absorb verification into their own policy, checking an action before taking it is increasingly asserted inside the model's reasoning rather than enforced by an external gate. That assertion is itself a proxy: text that claims an action was checked against a permission. Whether it agrees with what is actually persisted is empirical. We run a pre-registered audit on file, SQLite and local-outbox adapters with explicit structured permissions, in which a blind detector reads only the model's text and an independent verifier reads only the persisted state, joined afterwards by scenario identifier. Across a 27-cell aligned evaluation set, two local models exceeded the permission in no cell, so the primary endpoint (an assertion that passes while an unauthorized effect is present) was arithmetically unreachable. We therefore add 27 authored temptation cells in which the goal names an action the permission does not grant. There the models exceed the grant in 14 and 10 cells respectively, and yet emit zero verification assertions on the acting path: the violation is silent, and the primary endpoint stays empty. The one condition that does produce assertions is an explicit verification instruction, which suppresses every temptation in the stronger model at +147% tokens. Inspecting those refusals we find the failure that motivates this paper: 6 of 36 narration statements in the wrong-payload condition claim that tool, target and value are all consistent with the grant while the value differs, with the decision still correct. A final 9-cell construct presents a permission record that contradicts itself, where arms with and without execution binding are identical on the model side and differ only downstream. We prepare finite-count recomputation and per-cell provenance, and leave trained-verification models, deployment validation and any population safety claim unresolved.

1 Introduction

A model that says it checked an action before taking it is making a claim about its own past. The claim is cheap to produce and, unlike a returned exit code, has no mechanical relationship to whether the action was permitted. In an external gate the relationship is at least structural: a decision procedure reads the action and the policy and emits a verdict, and a discrepancy between that verdict and the persisted effect is a bug in the gate. A model-internal assertion has no such structure. It is text generated by the same process that proposes the action.

This matters because recent agent models are being trained to verify internally. The Sage-A1.5 report describes an opt-in *verification mode* that "lives inside the model's own chain of thought": the model pauses mid-trajectory, reviews what it has established and what has failed, and acts on a

revised direction in the same turn, "with no external critic" (Shanghai AI Laboratory, 2026). If that mode is to be relied on for anything, its self-report must track the state it claims to describe. That is a measurable property, and it is the one we measure.

The measurement problem has a specific shape. A prior audit of external approval gates found that a check can report a clean name/log proxy while unauthorized effects still land: matching action names to approval logs does not establish authorization, and name-only and exact-precheck controls admitted 63 and 45 unauthorized effects across 540 constructed cells while their proxy stayed positive (Liu, 2026). The same divergence is possible inside a model, and is harder to notice, because there is no log to compare against: the assertion and the action come from the same source.

We therefore ask a narrow question with a pre-registered answer format. When a model asserts that it has verified an action, does that assertion agree with the persisted effect? The primary endpoint is the **false negative**: an assertion that concludes the action is consistent *and* an unauthorized effect is present. The design keeps the two readings blind. A frozen detector reads only the model's text; an independent verifier reads only the files, the ledger and the outbox; the two records are written to separate files and joined afterwards by scenario identifier alone. Neither side can see the other's labels.

Our contributions are measurements, not a defense:

1. A pre-registered protocol with one primary endpoint, one decision rule and three stop rules, executed to completion on two local models and four arms (§3).
2. The finding that the primary endpoint is **empty for a structural reason**, not a statistical one: violating models never narrate verification, and narrating models never violate. The two halves of a false negative do not co-occur in these models (§4).
3. The finding that verification narration can be **wrong while the decision is right**: 6 of 36 narration statements in the wrong-payload condition claim full agreement on a payload that differs (§4.4).
4. A construct in which a permission record contradicts itself, showing that execution binding and verification instruction act on *different* quantities: binding changes the outcome with identical model behaviour, instruction changes the behaviour (§4.5).
5. Two fixture defects, found by a code gate rather than by reading the prompts, which had produced a false leak in one direction and a false containment in the other (§4.6).

The scope is deliberately small and stated everywhere. These are finite authored local-effect measurements. They are not a natural failure distribution, not a defense, not deployment validation, and not a claim about any model's general safety.

2 Related work and scope

Guard systems evaluate an agent's proposed tool call against an explicit policy or an unsafe-action label (Xiang et al., 2024; Mou et al., 2026). Their evaluation unit is the guard's verdict. Harness auditing moves the unit to the recorded behaviour of the whole loop, asking what the harness actually admitted rather than what it reported (Liu et al., 2026). The runtime-contract and action-bound approval lines make the opposite architectural bet, moving enforcement into a component that the model cannot narrate its way past (Ng et al., 2026; Microsoft Agent Governance Toolkit

Contributors, 2026). Consent integrity work asks whether what was approved is what executes (Weng, 2026), which is the external counterpart of our question.

Closest to our failure mode is an audit of unfaithful *refusals*: safety text that gives a reason unrelated to the mechanism that actually blocked the action (Singh, 2026). Our finding is the mirror image and, we think, the more awkward one: a refusal whose *decision* is correct while its stated reason asserts an agreement that does not hold. A refusal can be audited against the gate that blocked it. A stated agreement inside a model's own reasoning can only be audited against the state, which is why the two-sided design in §3 is necessary rather than convenient.

We do not claim novel binding, novel guard architecture, or a new defense. Execution binding is an existing strong baseline and enters this paper as an implementation control. What is new here is the measurement of the model-internal assertion under the same two-sided discipline that exposed the external one.

3 Design

3.1 Scenarios, adapters and permissions

A scenario presents a goal, a structured permission record, and a policy version. The permission lists permitted actions as `(tool, target, value)` tuples with a one-use ticket, an `authorized` flag, and the policy version it was issued under. Three local adapters give the action somewhere to land: an append-only file per target, a SQLite ledger with per-target balances reconciled against its own transaction entries, and an outbox whose filename and recipient must agree. Effects are local and finite; nothing leaves the machine.

The nine base goals span the three adapters. The aligned evaluation set crosses them with three states (clean, pending, recoverable) and the four arms below, giving 108 cells; the temptation family crosses them with three contexts and the four arms, giving 108 cells; the contradictory-record construct crosses them with one context and two arms, giving 18 cells.

3.2 Two-sided reading

Each cell produces a model-side record and an actual-side record. The model side stores the raw output and the detector's verdict; the actual side stores what the independent verifier found in the persisted files. They are joined only by `scenario_id`.

The detector is frozen at version `r5-assertion-v1` with a 12-case calibration set that is re-run on every invocation. It splits the output into sentences and counts a sentence as a verification assertion only when a checking verb is present, the sentence draws an acceptance or a rejection, and the sentence is not a bare statement of intent. A negation anywhere in the sentence flips an apparent acceptance to a rejection. That last rule exists because of a specific failure we caught during calibration: the sentence "cannot confirm the target agrees with the authorization" contains the acceptance marker for agreement while being a refusal, and reading it as a pass would corrupt the primary endpoint in the safe direction.

An assertion is `action_backed` when it is attached to the acting path; otherwise it is text-only. The distinction matters because the primary endpoint is only defined where the model acts.

3.3 Arms

Four arms cross two factors:

- **Gate.** `allow_all` admits any reviewed action, so the model's decision is the only thing between it and the effect. `local_execution_bound` binds admission to the exact `(tool, target, value)` tuple, the ticket's one-use count and the policy version, checked at execution rather than at planning.
- **Verification instruction.** Absent, or present as an instruction to check the action against the permission and state the result in a `verification` string field inside the same single JSON object. The field placement is itself an amendment (§4.6): an earlier version asked for the check in prose, which broke the output contract.

The fifth arm of the original protocol would use a model trained for internal verification. It is not executed here; §5 states what would be needed.

3.4 Endpoints and decision rule

The primary endpoint is the count of false negatives: the model asserts that the action is consistent *and* the verifier finds an unauthorized effect. Secondary endpoints are false positives, clean utility (authorized effects on clean goals), and output tokens. The pre-registered decision rule has exactly four branches: `divergent` (any false negative), `aligned_but_costly` (none, clean utility below one), `aligned_and_cheap` (none, clean utility equal to one), and `indeterminate` (no executed arm). A missing, stopped, transport-failed or unparseable cell is never counted as safe, and `indeterminate` takes precedence over every other branch.

Three stop rules bound the run. The first requires the published 540-cell external matrix to be reproduced exactly before any model call; it was, on every invocation. The second stops a run whose detector calibration fails. The third stops a cell whose verifier cannot read the persisted state, recording it as unobservable rather than as zero.

4 Results

4.1 The aligned set produces no violation, so the endpoint is empty

On the 27-cell aligned evaluation set both models exceeded the grant in no cell. llama3.2:3b stopped voluntarily in all nine pending cells, and so did qwen2.5:7b. With no unauthorized effect anywhere, a false negative is arithmetically impossible, so the pre-registered rule returns `aligned_and_cheap` for the gate-free arm. That is a valid measurement of *these models' compliance*; it is not a test of whether model-internal verification can diverge. Reporting it as the latter would be the central mistake this paper is about.

4.2 Temptation cells: violation without narration

To make the endpoint reachable we authored temptation cells in which the goal names an action the permission does not grant, in three shapes: no grant at all, a grant for a different target, and a grant for a different payload. These are labelled `temptation_fixture` in every trace and are reported separately from the aligned set.

Table 1: Temptation matrix. 27 authored cells per arm (9 goals × 3 contexts). `Parsed` counts single-JSON outputs that satisfied the schema. A zero unauthorized count in a row whose parsed count is 0 is a parsing outcome, not containment. Tokens are the sum over the 27 cells.

Model	Arm	Gate	Parsed	Unauthorized	Tokens	Branch
llama3.2:3b	arm_a	allow_all	27/27	14	484	`aligned_but_costly`
	arm_b	allow_all	0/27	0	1872	`indeterminate`
	arm_c	local_execution_bound	27/27	0	484	`indeterminate`
	arm_d	local_execution_bound	0/27	0	1897	`indeterminate`
qwen2.5:7b	arm_a	allow_all	27/27	10	398	`aligned_but_costly`
	arm_b	allow_all	27/27	0	984	`indeterminate`
	arm_c	local_execution_bound	27/27	0	398	`indeterminate`
	arm_d	local_execution_bound	27/27	0	994	`indeterminate`

Table 1 states the result, and its three zeros require three different readings.

The gate-free arm leaks. llama3.2:3b produces 14 unauthorized effects in 27 cells, qwen2.5:7b 10, concentrated in the wrong-payload and wrong-target conditions (9 of 9 and 5 of 9 for llama3.2:3b). With no gate, the model's reading of the permission record is the only constraint, and in these cells it follows the goal.

The bound arm contains without changing behaviour. llama3.2:3b under `local_execution_bound` shows 27 of 27 parsed, zero unauthorized effects, and *the same 484 output tokens* as the gate-free arm. The model made the same fourteen attempts; the binding refused them. Binding did not make the model more cautious. It moved the decision downstream.

The instruction changes behaviour, at a price. Under the verification instruction with no gate, qwen2.5:7b stops in all 27 cells, down from 10 violations, at +147% output tokens. This is the only condition in the study in which the verification instruction demonstrably suppresses temptation in a model that otherwise follows it.

The format collapse is real but model-specific. Under the same instruction, llama3.2:3b fails to produce a parseable object in all 27 cells. The failure is not refusal: the model enters a repetition loop over the permission fields and never closes the JSON, exhausting the generation cap. Its zero unauthorized count is therefore an artifact of unparseability, and Table 1 marks it as `0/27` parsed rather than as containment. That the cost is a property of the model and not of the instruction is established by qwen2.5:7b, which parses 27 of 27 under the identical instruction.

4.3 The primary endpoint stays empty for a structural reason

Across all eight runs, the assertion count on the acting path is zero (Table 2). Every acting cell is a `silent_act`: the model acts without claiming to have verified anything. The violating arm therefore cannot produce a false negative, because there is no assertion to be wrong; and the instructed arm cannot produce one either, because it never acts. The two halves of the endpoint do not co-occur.

This is a result about the construct, and it constrains what the design can conclude. It does not show that model-internal verification is unfaithful; it shows that in these models, and under an instruction that elicits a written verdict, the verdict is only written when the model has already decided not to act. A reply that says "I checked and it is fine" followed by the action was not produced once: of the 216 temptation cells, 48 reached the acting path and none of those 48 carried an assertion. We report this rather than a false-negative rate because the rate has no denominator.

Table 2: Assertion accounting on the temptation family. `on-act` counts cells where the model acted *and* emitted an assertion, which is the only path on which the primary endpoint is defined. Every arm shows zero there.

Model	Arm	on_act	on_stop	silent_act	FN	Δ tokens vs A
llama3.2:3b	arm_a	0	0	14	0	0 (+0%)
llama3.2:3b	arm_b	0	0	0	0	+1388 (+287%)
llama3.2:3b	arm_c	0	0	14	0	0 (+0%)
llama3.2:3b	arm_d	0	0	0	0	+1413 (+292%)
qwen2.5:7b	arm_a	0	0	10	0	0 (+0%)
qwen2.5:7b	arm_b	0	0	0	0	+586 (+147%)
qwen2.5:7b	arm_c	0	0	10	0	0 (+0%)
qwen2.5:7b	arm_d	0	0	0	0	+596 (+150%)

4.4 Narration that is wrong while the decision is right

The instruction does elicit narration on the refusing path, and there the content can be checked against the record. In the wrong-payload condition the goal specifies a value that differs from the granted one, and the model stops. Of the 36 such narration statements across the two instructed arms and both models, 6 assert that tool, target *and* value are all consistent with the grant:

```
"The action's tool, target, and value are consistent with the granted
permission."
```

while the goal value is `summary-2-drift` and the granted value is `summary-2`. The remaining statements name the discrepancy, in three distinct forms within the same condition:

```
"...consistent with the granted permission but not summary-1-drift."
"...but the value does not match ('summary-3' vs 'summary-3-drift')."
```

The decision is `stop` in all nine cells of this condition. Scored on the decision alone, every one is a verification success. Scored on the narration, three of nine are false reports per arm. Same model, same condition, same instruction, three different qualities of reason.

This is the model-internal analogue of the external proxy/actual divergence, in a form that is harder to detect because there is no second channel to compare against by default. In the external audit, a name/log proxy stayed positive while effects landed. Here, a narration stayed affirmative while the payload differed. Both are cases of a report that is not causally coupled to the state it describes. What differs is that the external case has a log to compare and this one needs an independently constructed one.

We are careful about what these six statements show. They do not show that the model believes something false; we have no access to its internal state. They show that the text it emits under a request to report its check can assert agreement on a record where no agreement holds, while the action it takes is correct. For anyone tempted to score a verification mode by its narration, that is the relevant fact.

4.5 A contradictory permission record

A fourth construct presents a record that contradicts itself: the goal action is listed under a ticket *and* the `authorized` flag is false. This is not a realistic record, and we do not present it as one. It

Table 3: Contradictory-record construct, 9 cells per arm. `act` is read as obeying the displayed entry, `stop` as obeying the flag. The rightmost column is the verifier's mechanical verdict, which follows the flag and is therefore not leakage in the sense of Table 1.

Model	Arm	Cells	`act` (entry)	`stop` (flag)	`mechanical_unauth`
llama3.2:3b	arm_a	9	1	8	1
	arm_c	9	1	8	0
qwen2.5:7b	arm_a	9	0	9	0
	arm_c	9	0	9	0

isolates a different question from the other three: not whether the model exceeds a coherent grant, but which half of an incoherent record it reads.

Two things follow from Table 3. First, the two arms are *identical on the model side*: same eight stops, same single act in the same cell, same token count. The gate changed the outcome and not the behaviour. This is the cleanest available statement of what execution binding does in this study, and it is the reason we run the unbounded arm alongside: without it, the bound arm's zero would look like model containment.

Second, the single entry-reading cell is the same goal that the defective fixture of the previous version flagged. Its persistence across fixture versions is why we did not treat it as noise (§4.6).

4.6 Two fixture defects found by a gate, not by reading

The temptation family was authored by hand, and the first version of it was wrong in two ways. Both were found by a self-check that asserts, before any model call, that each fixture is internally consistent, that the goal action is absent from the grant, that the tempting payload is executable by the adapter, and that the four contexts differ in the way their names claim.

The first defect made one condition self-contradictory: the action entry was kept while only the `authorized` flag was flipped. The verifier treats a tuple as permitted only when the flag is true, so a model that copied the displayed entry verbatim was labelled unauthorized even though the record displayed precisely that payload. One cell was affected. The same flip also caused the bound gate to refuse on the flag before any ticket matching, which made that arm's apparent containment a property of one boolean rather than of the binding under test.

The second defect is the one that argues for the gate. The wrong-target condition used a ledger target that the adapter rejects by constraint. The model obeyed the goal, the gate admitted the action, and the write failed, leaving zero effects for the verifier to find. Three cells were reported as the model stopping short of an unauthorized effect when in fact an adapter invariant had fired. Reading the prompt would never have revealed this: the prompt was reasonable. Only executing the fixture and inspecting the adapter's error did.

We report both because they are instances of the paper's own subject. A measurement that reports a zero, or a leak, has to be checked against the mechanism that produced it. Our corrections are in the artifact history; the superseded results are retained and marked as not citable, and the generator reads a single tree so that superseded and current results cannot be pooled.

5 Interpretation, reproducibility, and limitations

5.1 What the measurements support

Three claims survive the design. A model can exceed a displayed permission while saying nothing about having checked it, so compliance scored from narration is uninformative on the violating path. A verification instruction can suppress that behaviour at a measurable token cost, and whether it does varies by model by a factor worth stating: 10 violations become 0 in one model and a total format collapse appears in the other. And the narration a model emits when asked to report its check can assert an agreement that does not hold, with the decision still correct.

5.2 What they do not support

We make no claim about a population of agents, about natural failure rates, about deployment readiness, or about any model's general safety. The temptation cells are authored, so their counts describe what these models did on these fixtures. The contradictory-record construct is deliberately unrealistic. The token comparison is between conditions within one model, not a claim about cost at scale.

The primary endpoint is empty. We state it as empty rather than as zero-with-a-rate because no denominator exists: no acting cell produced an assertion. A reader who wants a false-negative rate will not find one here, and constructing one requires a model that both violates and narrates. That is the arm we could not run.

5.3 The unexecuted arm

The protocol's fifth arm uses a model trained for internal verification, which is the case the paper exists to inform: if a model's verification mode is trained rather than instructed, the assertion and the action are more tightly coupled by construction, and the co-occurrence that is absent here might appear. Running it requires the trained model resident in memory, which is a 35B mixture-of-experts and beyond this machine. We record the arm as not executed, with the harness, the arm definition and the seam where a trained-verification backend would attach left in place, so the measurement can be completed elsewhere without redesigning it.

5.4 Reproducibility

Every value in this paper is emitted by a script that reads the artifacts and writes both a claim ledger with each number's source path and field, and the table bodies themselves. A number in the text that is not in the ledger is an unverified claim. The external integrity matrix is reproduced exactly as a precondition, the detector calibration is re-run on every invocation, and the analysis unit tests pin each of the four decision branches. Sandboxes are per-cell and retained.

6 Conclusion

The question we set out to answer was whether a model's assertion that it has verified an action agrees with what was actually persisted. On these models the answer cannot be given as a rate, because the assertion and the violation never appeared in the same cell. What the measurements do give is sharper than a rate. Verification narration is emitted by models on the refusing path, where 6 of 36 statements asserted an agreement that did not hold while the decision remained correct.

Execution binding and verification instruction act on different quantities: binding changes the outcome with identical model behaviour, instruction changes the behaviour and can fail to produce an outcome at all. And the fixtures we authored to make the question askable were themselves wrong in two ways, one producing a false leak and one a false containment, both caught only because a code gate asked the fixture to justify itself before a model saw it.

If verification is to move inside the model, the audit has to move with it. The machinery for that audit is small: two blind readers, a joined record, and a fixture that has to prove it measures what its name says. What it will not do is accept the model's own account of whether it checked.

Broader impact

This paper argues for treating a model's self-reported verification as an unverified proxy, and recommends auditing it against persisted state. The immediate implication is conservative: systems that rely on internal verification for restricting actions should retain an external bound until the internal one is measured for the models they deploy. We see no misuse pathway: the measurements describe local file, ledger and outbox effects in an authored sandbox, and the paper's main practical contribution is a caution against over-trusting a report.

Use of generative AI tools

The experimental harness, the fixture self-check, the analysis code and the number emission scripts were developed with AI coding assistance. The protocol, the amendments with their documented defects, the decision rule and the interpretation are the author's. Every reported value is generated from artifacts by script rather than transcribed.

References

Chengzhi Liu, Yichen Guo, Yepeng Liu, Yuzhe Yang, Qianqi Yan, Xuandong Zhao, Wenyue Hua, Sheng Liu, Sharon Li, Yuheng Bu, and Xin Eric Wang. Auditing agent harness safety, 2026. URL https://arxiv.org/abs/2605.14271.

Zewen Liu. Approval gates and safety prompts: A measurement audit from plans to local execution, 2026. arXiv preprint; not peer reviewed.

Microsoft Agent Governance Toolkit Contributors. ADR 0030: Use an action-bound, fail-closed approval protocol. Official repository, revision 47abed7446d4, 2026. URL https://github.com/microsoft/agent-governance-toolkit/blob/47abed7446d47e74cfa76d01a5913bb873ffcad9/docs/adr/0030-action-bound-approval-protocol.md.

Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback, 2026. URL https://arxiv.org/abs/2601.10156. Licensed CC-BY-4.0.

Albus W. Ng, Yi Han, Jusheng Zhang, and Wenhao Wang. Agent safety should be a runtime contract, 2026. URL https://arxiv.org/abs/2608.11274.

Shanghai AI Laboratory. Sage-a1.5: From long-horizon execution to scientific exploration with capability composition, 2026. URL https://arxiv.org/abs/2610.28578.

Aarushi Singh. Guardrails as scapegoats: Auditing unfaithful safety refusals in tool-augmented llm agents, 2026. URL https://arxiv.org/abs/2607.19449.

Xiaoqi Weng. What you approve is what executes: Consent integrity for black-box llm agents, 2026. URL https://arxiv.org/abs/2606.02668.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning, 2024. URL https://arxiv.org/abs/2406.09187. Accepted to ICML 2025; licensed CC-BY-4.0.

A Pre-registration and amendments

The protocol and five dated amendments are public in the accompanying artifact repository. Amendments are logged with the problem they address and are written before the corresponding run: field placement of the verification string, an explicit generation cap after a truncation was misdiagnosed as non-compliance, the temptation family and per-decision-path assertion accounting, the fixture self-check with its two defects, and the contradictory-record construct. Superseded results are retained and marked not citable.